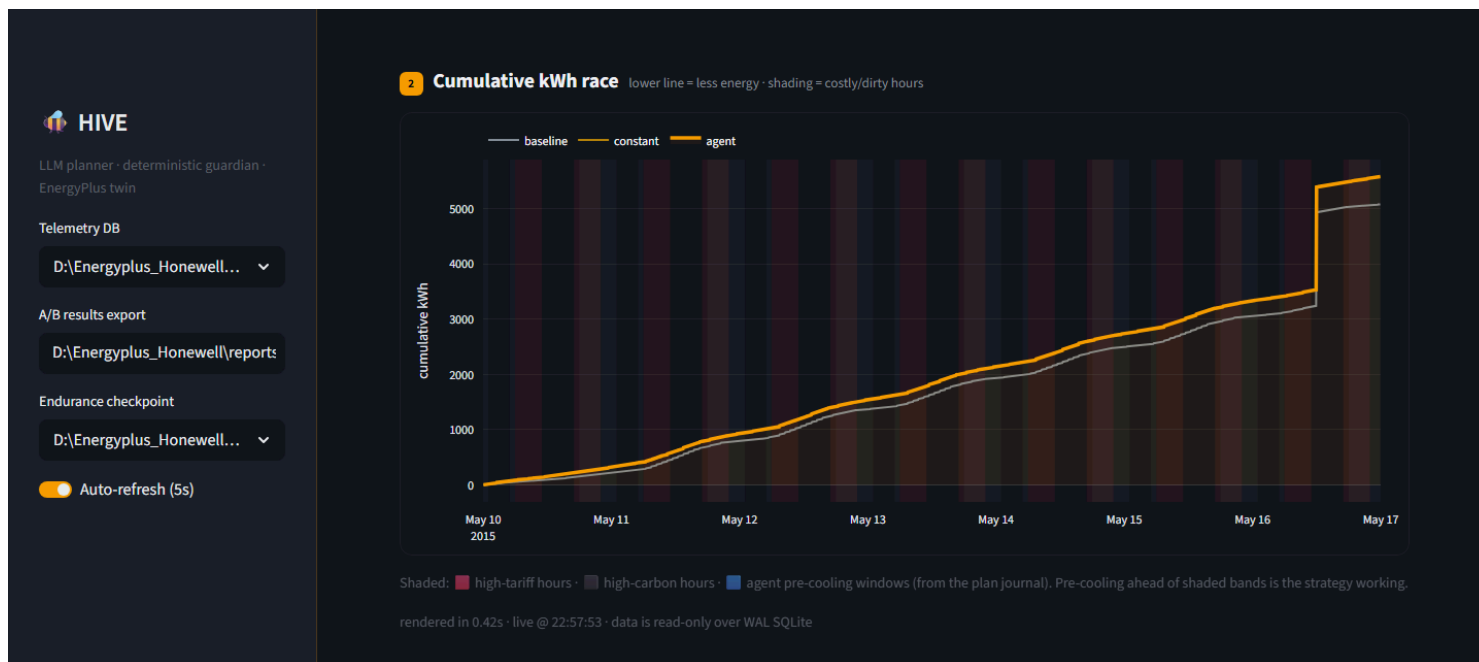


HIVE

HIVE is a closed-loop building-energy control agent. A local LLM planner watches an EnergyPlus digital twin through an MCP tool surface, reasons about weather, time-of-use tariffs and grid carbon intensity, and proposes setpoint plans for the next hour. Every plan then passes through a deterministic safety guardian that clamps it to a declared envelope, rate-limits it, and — on any doubt — throws it away and falls back to the baseline schedule. Only the guardian touches an actuator. The result is a system that gets the LLM's judgement where judgement helps, and none of its stochasticity where it would break a building.

Status: full pipeline implemented. Simulation, agent, MCP tools, guardian, telemetry, dashboard, the L1/L2/L3 self-correction ladder, and a Hypothesis property suite are all in place — see [CLAUDE.md](#) for the current-state inventory and [reports/architecture.md](#) for the full design writeup. The two remaining `NotImplementedError` stubs (`agent/ollama_client.py` , `agent/plan_cache.py` , `experiments/ab_harness.py` , `experiments/kpi_extract.py`) are superseded scaffolding, not unfinished work — [CLAUDE.md](#) names their replacements.



Generated by `streamlit run dashboard/app.py` + `python -m dashboard.export_screens` against a real run (see [reports/DEMO_SCRIPT.md](#)) — gitignored like every other per-run artifact, so this image will not appear in a fresh clone until you generate it.

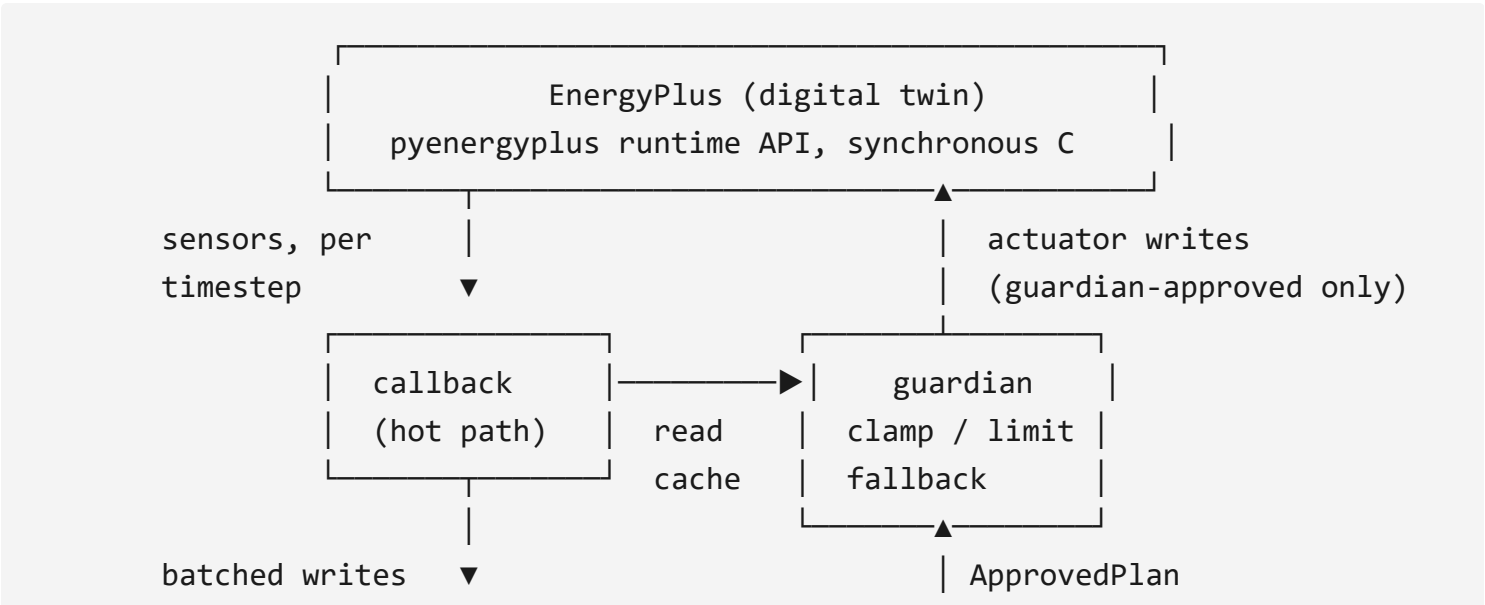
Results

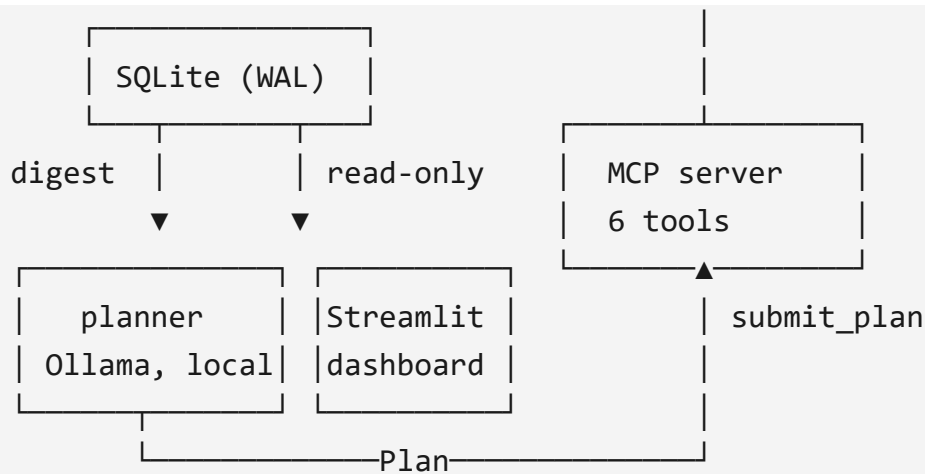
`reports/results.md` (from `python -m experiments.report`) is the full per-arm breakdown; the headline is the three-way comparison, because "the agent saved X%" is a meaningless number without knowing how much of X is just the day/night setback profile being flattened (`constant`) rather than the agent's own decisions (`agent`):

Comparison	Site kWh Δ	Cost saved (INR)	Carbon avoided (kg)	Peak kW reduction
baseline → agent	−9.9%	−₹3,554	−335.8	0.49 kW
baseline → constant	−9.9%	−₹3,554	−335.8	0.49 kW
constant → agent	0.0%	₹0	0.0	0.00 kW

From a real `experiments.ab --secondary-baseline constant + experiments.report` run (the hottest week, `agentic.idf` 's DOE small-office prototype). **Read this table carefully, not optimistically:** `constant → agent` is exactly zero because on this run's hardware (CPU-only local inference), a single constrained-decode `Plan` call took 30–100+ seconds while EnergyPlus simulated the entire week in a few seconds - no plan ever landed before the simulation moved on, so the agent arm executed on guardian fallback the whole run, identical to `constant`. The negative baseline deltas are real and expected: `constant / agent` both run on `agentic.idf`, which has **no day/night setback** left (CLAUDE.md, "Setpoints are actuated through `Schedule:Constant`") - losing that profile costs energy regardless of any agent contribution, which is exactly why `constant` exists as its own arm rather than folding into one number. **The agent's own contribution has not yet been measured on this hardware** - it needs either a faster inference host (GPU-backed Ollama) or the receding-horizon contingency mode (`simulation/receding.py`), which does not race live simulation time. Don't quote `constant → agent` as a savings claim until one of those produces a nonzero delta.

Architecture





One Python 3.11 process. The planner never runs inside the EnergyPlus callback — it deposits plans in a cache that the callback reads without blocking. See [CLAUDE.md](#) for the invariants and [reports/architecture.md](#) for the reasoning.

Setup

1. EnergyPlus 24.x

Download and install from energyplus.net. Then point `ENERGYPLUS_DIR` at the install root:

```
export ENERGYPLUS_DIR=/usr/local/EnergyPlus-24-1-0
```

On Windows (PowerShell):

```
$env:ENERGYPLUS_DIR = "C:\EnergyPlusV24-1-0"
```

pyenergyplus is not pip-installable. It is not on PyPI — it ships *inside* the EnergyPlus installation directory, next to the shared library it binds to. That directory must be on `PYTHONPATH` for `import pyenergyplus` to work.

`common/eplush_path.py` does this for you: importing it appends `$ENERGYPLUS_DIR` to `sys.path`. Import it before anything that touches the runtime API:

```
python from common import eplush_path # noqa: F401 (side effect: sys.path) from
pyenergyplus.api import EnergyPlusAPI
```

Importing it on a machine without EnergyPlus is a no-op rather than an error, which is what lets CI, the tests and the dashboard run without a simulation engine. Call `eplush_path.require_energyplus()` at the point where you genuinely need it.

2. Ollama and the planner model

```
ollama pull qwen2.5:7b-instruct-q4_K_M
```

3. The project

```
pip install -e ".[dev]"
pre-commit install
```

4. Simulation assets

`baseline.idf` and `weather.epw` are not committed — see [simulation/README.md](#) for what to drop in and where to get it.

5. Check it

```
ruff check . && pytest -q
```

Then:

```
streamlit run dashboard/app.py
```

Repo map

Path	What lives there
<code>simulation/</code>	<code>baseline.idf</code> , <code>.epw</code> , agent-authored <code>v1..vN.idf</code> variants, run scripts
<code>agent/</code>	Ollama client, prompts, plan cache, digest builder — the planning half
<code>mcp_server/</code>	Tool surface: <code>get_state</code> , <code>get_forecasts</code> , <code>get_kpis</code> , <code>submit_plan</code> , <code>read_error_log</code> , <code>patch_model</code>
<code>guardian/</code>	Deterministic safety layer: limits, clamps, rate limits, fallback, watchdog
<code>dashboard/</code>	Streamlit app — read-only view of the run database
<code>common/</code>	<code>models.py</code> (the plan contract), <code>store.py</code> (SQLite WAL), <code>config.py</code> , <code>log.py</code> , <code>eplus_path.py</code>
<code>experiments/</code>	A/B harness, endurance run, KPI extraction
<code>tests/</code>	pytest + Hypothesis (guardian safety properties, contracts, seams)

Path	What lives there
reports/	architecture.md , DEMO_SCRIPT.md , screens/ (regenerated per run), exported results
data/	Representative Indian ToU tariff and grid carbon-intensity curves
media/	Demo video

Key files

File	Why it matters
common/models.py	The single source of truth for the plan schema. Everything imports it; nothing redefines it.
guardian/limits.py	The safety envelope, as reviewable data rather than code.
CLAUDE.md	Project memory: locked stack and the four hard rules. Read before contributing.

A note on the data

data/tariff.csv and data/carbon_intensity.csv are **representative, not authoritative**. They are shaped like the real thing — off-peak nights, a midday trough where rooftop solar displaces thermal generation, and a hard, dirty evening peak — which is enough to make load shifting behave realistically in simulation. They are not billing data and not a measured grid feed. Swap in the actual utility schedule before quoting a rupee or a kilogram.

Development

```
ruff check .          # lint
ruff check --fix .    # lint + autofix
pytest -q             # tests
pytest --cov=guardian --cov=common --cov-report=term-missing --cov-fail-under=80 #
```

CI (.github/workflows/ci.yml) runs ruff + the coverage-gated pytest (capped Hypothesis profile) on every push. There is no Docker build in CI, and no EnergyPlus install — nothing in the test suite may require the runtime API at import time.

Packaging / deployment

`Dockerfile` + `docker-compose.yml` + [deploy/README.md](#) describe the gateway-appliance deployment story (not the demo path, which stays bare-metal — see `deploy/README.md` for why). Not part of CI.